CorpusLab Technical Handbook

Product Overview

CorpusLab is a corpus workbench for teams building retrieval-augmented generation systems. The product helps engineers inspect sources, extracted documents, chunks, embeddings, retrieval runs, trace timelines, evidence summaries, evals, and reports across arbitrary knowledge sources: product docs, support knowledge bases, policies, contracts, research papers, technical specifications, code documentation, wikis, and resumes.

The current implementation is privacy-first and local by default. Uploaded binaries are not persisted. Extracted text, chunk text, metadata, embeddings, retrieval runs, traces, evals, and reports are stored in Postgres. Retrieval and trace diagnosis use local lexical scoring and a local hash embedding baseline so relevance can be debugged without external model calls.

Repository Structure

- `apps/api`: Axum API service, runtime config, routes, startup state, telemetry, and integration tests.
- `apps/web`: React, Vite, TypeScript workbench UI with Overview, Sources, Retrieval, Traces, Evals, Reports, and Settings pages.
- `crates/core`: shared contracts for projects, sources, documents, chunks, ingestion, embeddings, retrieval, traces, evals, reports, and config.
- `crates/rag`: extraction, chunking, document intelligence, embeddings, retrieval scoring, trace construction, and eval metrics.
- `crates/storage`: repository traits plus in-memory and SQLx/Postgres storage implementations.
- `migrations`: SQLx migrations for projects, sources, documents, chunks, embeddings, retrieval runs, traces, and evals.
- `docs`: architecture, development, testing, privacy, ingestion, retrieval, traces, Eval Lab, roadmap, ADRs, and this handbook.

Rust Workspace Architecture

The Rust workspace keeps domain contracts separate from implementation. `crates/core` owns serializable types and IDs. `crates/rag` owns deterministic RAG behavior. `crates/storage` owns persistence boundaries. `apps/api` composes those crates into HTTP routes.

This shape lets future hosted services, local collectors, workers, and GPU/HPC indexing processes reuse the same contracts without coupling them to Axum route code.

React App Architecture

The web app lives in `apps/web/src`.

- `App.tsx` maps the public site, auth pages, and workbench routes.
- `layouts/MarketingLayout.tsx`, `layouts/AuthLayout.tsx`, and `layouts/WorkbenchLayout.tsx` separate public, authentication, and application chrome.
- `features/marketing` owns the landing page, feature page, pricing page, and launch-state product copy.
- `features/auth` owns the login and signup entry surfaces.
- `components/brand` owns the CorpusLab mark and wordmark.
- `components/ui` owns reusable marketing and product primitives such as buttons, feature cards, pricing cards, and product mockups.
- `lib/apiClient.ts` is a compatibility barrel for the API boundary. New code should prefer domain exports under `lib/api`, such as `lib/api/sources`, `lib/api/retrieval`, `lib/api/traces`, and `lib/api/evalLab`.
- `pages/OverviewPage.tsx` summarizes corpus readiness under `/app`.
- `pages/SourcesPage.tsx` handles upload, ingestion results, document tables, and chunk preview under `/app/sources`.
- `pages/RetrievalPage.tsx` handles query, filters, embedding indexing, evidence summary, grouped hits, score bars, save-as-trace, and save-to-eval under `/app/retrieval`.
- `pages/TracesPage.tsx` handles trace list, timeline spans, ranked evidence, failure labels, rerun comparison, and in-app explainers under `/app/traces`.
- `pages/EvalsPage.tsx` handles Eval Lab datasets, cases, experiment runs, mode comparison, failure diagnosis, and gates under `/app/evals`.
- `pages/ReportsPage.tsx` and `pages/SettingsPage.tsx` complete the workbench pages.

Generated `apps/web/dist` files should not be edited by hand. Run `cd apps/web && npm run build`.

API Route Reference

- `GET /healthz`: process liveness.
- `GET /readyz`: readiness; checks database connectivity when storage is configured.
- `GET /api/v1/config`: safe product/runtime config for the web app.

- `POST /api/v1/sources/files`: multipart ingestion for text, Markdown, HTML, and PDF.
- `GET /api/v1/sources`: sources with document and chunk counts.
- `GET /api/v1/documents/:document_id/chunks`: persisted chunks ordered by ordinal.
- `GET /api/v1/embeddings/status`: embedding model and indexing coverage.
- `POST /api/v1/embeddings/index`: synchronously indexes stored chunks with the local provider.
- `POST /api/v1/retrieval/query`: lexical, vector, or hybrid retrieval with evidence summary and citations.
- `GET /api/v1/traces`: recent trace summaries.
- `GET /api/v1/traces/:trace_id`: full trace timeline.
- `POST /api/v1/traces/from-retrieval-run`: save a retrieval run as a trace.
- `POST /api/v1/traces/:trace_id/rerun`: rerun a trace with changed retrieval settings.
- `GET /api/v1/retrieval/evals`: saved eval cases.
- `POST /api/v1/retrieval/evals`: create an eval case.
- `POST /api/v1/retrieval/evals/run`: run eval cases for a retrieval mode.
- `GET /api/v1/eval-lab/datasets`: list Eval Lab datasets.
- `POST /api/v1/eval-lab/datasets`: create an Eval Lab dataset.
- `GET /api/v1/eval-lab/datasets/:dataset_id`: load a dataset with cases.
- `POST /api/v1/eval-lab/datasets/:dataset_id/cases`: add a case to a dataset.
- `PATCH /api/v1/eval-lab/cases/:case_id`: update a case.
- `DELETE /api/v1/eval-lab/cases/:case_id`: delete a case.
- `POST /api/v1/eval-lab/experiments`: run a dataset across selected retrieval modes.
- `GET /api/v1/eval-lab/experiments`: list recent experiments.
- `GET /api/v1/eval-lab/experiments/:experiment_id`: load one experiment.
- `POST /api/v1/eval-lab/experiments/:experiment_id/compare`: compare selected modes.

Database Schema And Migrations

Migrations are in `migrations`.

Core tables include `projects`, `sources`, `ingestion_runs`, `documents`, `chunks`, `chunk_embeddings`, `retrieval_playground_runs`, `retrieval_playground_hits`, `debug_traces`, `trace_rerun_experiments`, `retrieval_eval_datasets`, `retrieval_eval_cases`, `retrieval_eval_runs`, `retrieval_eval_results`, and `retrieval_eval_experiments`.

Recent metadata additions:

- `documents.document_profile`: one of `general`, `technical_docs`, `policy_or_legal`, `support_kb`, `research_paper`, `code_docs`, or `resume`.

- `documents.extraction_quality`: `high`, `medium`, `low`, or `unknown`.
- `documents.warnings`: extraction and quality warnings.
- `chunks.quality_flags`: heading-only, too-short, duplicate, low-density, extraction-warning, and evidence-candidate flags.
- `chunks.text_density`, `chunks.is_duplicate`, and `chunks.evidence_score_hint`.

Run migrations with `just db-migrate`.

File Ingestion Pipeline

The browser sends multipart files to `POST /api/v1/sources/files`. The API validates file count, per-file bytes, total request bytes, and extension allowlist using typed runtime config. The first workflow is synchronous so behavior is simple to observe.

For each file:

- Detect a supported format.
- Extract readable text with the Rust extractor.
- Compute file checksum.
- Detect document profile and extraction quality.
- Chunk text with structured document chunking or whitespace fallback.
- Annotate chunk quality and duplicate signals.
- Persist source, ingestion run, document, and chunk metadata.
- Return document results and preview chunks.

Original binaries are not stored.

Extraction Pipeline

`crates/rag/src/extraction.rs` owns extraction.

- Text and Markdown use UTF-8 extraction.
- HTML uses best-effort readable text extraction.
- PDF uses embedded text extraction through `pdf-extract`.
- OCR is intentionally not included yet.

Failures are structured as unsupported type, invalid UTF-8, PDF extraction failure, empty text, size limit, or storage failure.

Chunking Strategies

`structured` is the default. It generalizes the earlier resume-focused strategy into a document-aware structured chunker.

Structured chunking:

- Detects headings in technical docs, policies, support KBs, research papers, code docs, resumes, and general documents.
- Preserves paragraph and bullet group boundaries where possible.
- Packs blocks up to `target_tokens`.
- Uses whitespace fallback when one block exceeds the token limit.
- Applies overlap only for token-limit splits within the same section.

`whitespace` remains available for debugging and deterministic fallback comparisons.

Document Intelligence

`crates/rag/src/intelligence.rs` adds deterministic document analysis.

Profiles:

- `general`
- `technical_docs`
- `policy_or_legal`
- `support_kb`
- `research_paper`
- `code_docs`
- `resume`

Chunk quality flags identify weak evidence candidates before retrieval ranking promotes them. Heading-only chunks and duplicates are visible in the UI and suppressed from strong evidence summaries.

Embedding Provider System

`crates/rag/src/embedding.rs` defines the local embedding provider boundary. The current provider is `local-hash-v1` with configurable dimension. It is deterministic, local, and CPU-friendly. It is not intended to be the final semantic model.

The provider boundary is intentionally ready for future implementations:

- local ONNX embedding model
- CUDA worker
- Metal worker
- remote enterprise embedding service
- batch indexing queue

Retrieval Scoring

`crates/rag/src/retrieval.rs` implements the local baseline retriever.

Modes:

- `lexical`: term overlap, phrase matches, section boosts, path boosts, and metadata boosts.
- `vector`: cosine similarity over stored embeddings.
- `hybrid`: semantic similarity plus lexical and metadata signals.

The response includes raw and normalized score breakdowns:

- semantic
- lexical
- phrase
- section
- path
- metadata

Quality upgrades:

- Deduplicates hits by checksum and normalized text.
- Tracks `duplicate_count`.
- Adds `quality_flags`.
- Adds `evidence_strength`.
- Avoids promoting heading-only and weak chunks as strong answer evidence.
- Produces an evidence summary with cited snippets instead of pretending to generate unsupported claims.

Trace Debugger

`crates/rag/src/tracing.rs` converts a saved retrieval response into an inspectable trace.

Trace records include:

- query input
- retrieval mode and latency
- ordered spans
- ranked evidence and citations
- deterministic failure labels
- rerun comparisons

Current spans are query input, retrieval ranking, evidence summary, eval check, and optional generation metadata. The Eval Check span is present even before eval linkage so the product can teach users where regression checks fit in the workflow.

Failure labels include missing documents, missing embedding index, bad embedding, weak evidence, bad ranking, duplicate evidence, heading-only evidence, and bad chunking. These labels are deterministic quality signals derived from retrieval response metadata.

Postgres stores trace summaries in `debug_traces`, full trace timelines in `trace_json`, and rerun comparisons in `trace_rerun_experiments`. The memory store supports the same API for tests and local no-Docker sessions.

The web UI exposes `/app/traces` with a saved trace list, timeline, ranked evidence, rerun lab, and explainer cards. The Retrieval page exposes `Save trace` after a query completes.

Eval System

Eval Lab is the primary quality-control workflow. Datasets group cases by product area, customer workflow, release gate, or corpus. Cases store a query plus expected chunk IDs, document IDs, or both. Experiments run a dataset across selected retrieval modes and freeze a config snapshot with `top_k`, scoring weights, embedding model metadata, and dataset case count.

Metrics include recall@k, precision@k, MRR, top hit rank, citation coverage, weak evidence count, missing embedding failures, and latency p50/p95. Failure labels include expected evidence missing, correct document wrong chunk, low precision, weak evidence, missing embeddings, heading-only evidence, and duplicate evidence.

The default gate passes when average recall@k is at least `0.80`, critical missing-embedding failures are absent, and weak-evidence cases are at or below 20%. Failed gates appear in Mission Control and point users to `/app/evals`.

The legacy `/api/v1/retrieval/evals` endpoints remain compatible for older flows. Retrieval and Trace Debugger now save cases directly into Eval Lab datasets.

Report Contracts

`crates/core/src/report.rs` defines report contracts:

- `RetrievalReport`
- `RetrievalDiagnosis`
- `EvidenceIssue`

The UI now has a Reports page for corpus diagnostics. A future API route can persist and export full retrieval-run reports using these contracts.

Privacy And Security Model

Default behavior is local and privacy-first.

- Uploaded binaries are not stored.
- Extracted chunk text is stored in Postgres.
- Embeddings are stored in Postgres.
- No hosted LLM or hosted embedding API is called in the current retrieval path.
- `/api/v1/config` exposes safe config only; database URLs and deployment secrets stay server-side.

Hosted mode will need auth, tenant isolation, audit events, key management, upload scanning, and configurable data retention.

Configuration Model

`crates/core/src/config.rs` defines:

- `ProductConfig`
- `IngestionConfig`
- `ChunkingConfig`
- `RetrievalConfig`
- `RetrievalWeights`
- `EmbeddingConfig`
- `UiConfig`

`apps/api/src/config.rs` loads environment values, validates numeric fields, and exposes safe config through `GET /api/v1/config`.

All major defaults should be changed through `.env.example` values rather than hidden route constants.

Testing Strategy

Rust:

```
cargo fmt --check
cargo clippy --workspace --all-targets -- -D warnings
cargo test --workspace
```

Web:

```
cd apps/web
npm run typecheck
npm run lint
npm test
npm run build
npm run format:check
```

Full local check:

```
just check
```

Handbook PDF:

```
just docs-pdf
```

Roadmap To Hosted Team Product

The architecture should grow toward:

- Organizations and workspaces.
- Users, roles, and invitations.
- API keys and scoped service accounts.
- Local collector for private networks.
- Hosted control plane for dashboards, reports, and team collaboration.
- Worker queue for ingestion, embedding, OCR, reranking, and reports.
- Audit events for uploads, queries, reports, and config changes.
- Shareable reports with redaction controls.

Roadmap To GPU And HPC Workers

The future HPC path should stay cleanly separated from the API route layer:

- Add a worker crate for embedding and reranking jobs.
- Introduce job queues and worker leases.
- Add ONNX/Candle model loading for local embeddings.
- Add CUDA and Metal provider implementations behind the embedding provider trait.
- Add batch embedding throughput metrics.
- Add corpus-scale indexing benchmarks.
- Add vector index backends such as pgvector, Tantivy, or a dedicated ANN service.

The goal is not just faster retrieval. The goal is explainable retrieval quality at corpus scale.